

MIT 805: Part 2

Thigan Pillay
18106545

R Smit
18131451

Contents

1	Introduction	1
2	Processing Objective & Central Question	1
3	MapReduce-Style Implementation in PySpark	2
3.1	Mapping/Transformation	2
3.2	Shuffle/Grouping	2
3.3	Reduction/Aggregation	2
4	Spark Transformations vs Actions	3
5	Distributed Execution Evidence & Spark DAG	3
5.0.1	Reading the Physical Plan	4
5.0.2	Spark DAG Execution vs Hadoop MapReduce	4
6	Results & Technical Interpretation	5
6.1	Demand by Day of Week and Hour	5
6.2	Peak Hour Pickup Zones	5
6.3	Monthly Trip Volume	5
7	Business & Societal Interpretation	6
8	Conclusion	7
9	References	7

List of Figures

1	Trip counts by day of week and hour, computed via <code>demand_by_day_hour</code> (Section 4c). The darkest cells show that demand is highest during weekday evenings and around midnight on weekends.	8
2	The ten pickup zones with the most trips during the 5pm–7pm peak (Section 4b). LaGuardia Airport leads.	8
3	Total trip volume across the 14 evenly sampled months. The sharp drop around May 2020 corresponds to the COVID-19 pandemic’s effect on NYC transportation demand, supporting Part 1’s finding at full distributed scale. .	9

List of Tables

1	First five rows after the Map step	2
2	Excerpt of demand by day and hour	3
3	Top five pickup zones by trip count during the 5pm–7pm peak.	5
4	Selected monthly trip volumes showing the COVID-19 decline and subsequent recovery.	6

1 Introduction

Part 1 of this project focused on understanding and exploring the NYC Taxi and Limousine Commission's High Volume For Hire Vehicle (HVFHV) trip dataset spanning from February 2019 to May 2025 [1, 2]. The raw dataset comprises 73 monthly Parquet files totalling 30.21 GB. The complete working dataset contains 1,283,180,517 rows and is approximately 25 GB and the data preparation and quality analysis steps used to create the working dataset are detailed in Part 1 [3]. A 5% sample (64,159,022 rows), was analysed using Pandas to make exploratory analysis manageable on a single machine. This helped with understanding the data and pattern identification, but it does not demonstrate distributed computing of large datasets. Part 2 addresses this by using PySpark to process 245,504,968 rows (± 5 GB) directly from disk. This is almost 4 times larger than the sample used in Part 1 and demonstrates the use of distributed computing.

This analysis uses Apache Spark's DataFrame API and operations such as `withColumn`, `groupBy` and `agg`. Spark automatically handles the Map, Shuffle, and Reduce stages required to perform the DataFrame operations, rather than coding them manually. Spark's `.explain` method is used to examine the query plan and demonstrate how the Map, Shuffle and Reduce stages are carried out during distributed processing. The report also defines the processing objective, demonstrates the Map/Shuffle/Reduce processing pipeline, and provides the resulting analysis and visualisations together with their business interpretation.

2 Processing Objective & Central Question

Central question: How does HVFHV trip demand vary across different times of the week, and what do these patterns indicate about periods of high and low transport demand in New York City?

This question was selected because understanding when trip demand is highest provides practical value. These insights can be used to determine when drivers may be needed most, while transport authorities such as the NYC Taxi and Limousine Commission can use them to better understand where and when travel demand is at its peak.

Three analyses are examined to answer the central question:

1. Trip counts, average trip distance and duration are broken by day of the week and hour of the day to identify periods of high and low demand.
2. The most common pickup zones during peak demand is examined to determine where demand is concentrated.
3. Monthly trip volumes are also analysed to show how demand shifts over time, including the decline and recovery associated with COVID-19.

Together, these analyses provide a clear picture of when and where HVFHV demand is concentrated.

3 MapReduce-Style Implementation in PySpark

This project makes use of Spark's DataFrame API instead of the RDD API. The DataFrame API performs the same Map, Shuffle and Reduce operations using commands such as `withColumn`, `groupBy` and `agg`. Spark's Catalyst optimiser automatically converts these commands into an optimised distributed execution plan [4]. This is discussed further in Section 5.

3.1 Mapping/Transformation

Three new time based columns are created from *pickup_datetime* using `withColumn`. These transformations are applied to each row:

```
from pyspark.sql.functions import hour, date_format
df = df.withColumn("pickup_hour", hour("pickup_datetime"))
df = df.withColumn("pickup_day_of_week", date_format("pickup_datetime", "EEEE"))
df = df.withColumn("pickup_year_month", date_format("pickup_datetime", "yyyy-MM"))
```

The `hour()` function extracts the pickup hour from 0 to 23, while `date_format()` is used to get the day of the week and the year-month values. Since these values are calculated separately for each row, Spark does not need to combine or move data between partitions. Therefore these are classified as narrow transformations. Table 1 shows the Map step output.

Table 1: First five rows after the Map step

pickup_datetime	pickup_hour	pickup_day_of_week
2019-02-01 00:05:18	0	Friday
2019-02-01 00:41:29	0	Friday
2019-02-01 00:51:34	0	Friday
2019-02-01 00:03:51	0	Friday
2019-02-01 00:09:44	0	Friday

3.2 Shuffle/Grouping

The `groupby()` operation is a wide transformation because rows containing the same *pickup_day_of_week* and *pickup_hour* may be in different partitions. Spark needs to bring all rows with the same values together (shuffle) in order to calculate each day and hour combination. Once done, Spark can perform the required aggregation. This principle is applied to the `groupBy("PULocationID")` in Section 6.

3.3 Reduction/Aggregation

After the rows are organized by day and hour, `.agg()` computes three aggregated measures for each group namely, the total trip count, the mean trip distance, and the mean trip duration.

```
from pyspark.sql.functions import count, avg, round
demand_by_day_hour = (df.groupBy("pickup_day_of_week", "pickup_hour")
    .agg(count("*").alias("trip_count"),
```

```
round(avg("trip_miles"), 2).alias("avg_trip_miles"),
round(avg("trip_time"), 0).alias("avg_trip_time_seconds"))
.orderBy("pickup_day_of_week", "pickup_hour"))
```

In MapReduce terminology, this is the Reduce step, since numerous separate trip entries are merged into a more compact summary. Table 2 shows some of the results where trips are grouped into one row per hour. This makes it easier to identify high and low demand patterns and compare the average trip distance and duration across those intervals, as described in Section 6.

Table 2: Excerpt of demand by day and hour

<code>pickup_day_of_week</code>	<code>pickup_hour</code>	<code>trip_count</code>	<code>avg_trip_miles</code>	<code>avg_trip_time_seconds</code>
Friday	0	1,249,353	5.42	1043.0
Friday	2	532,469	5.28	937.0
Friday	3	400,450	5.98	970.0
Friday	8	1,993,500	4.43	1157.0
Friday	12	1,572,067	4.84	1247.0
Friday	17	2,182,583	4.31	1352.0
Friday	18	2,389,100	4.25	1236.0
Friday	19	2,484,465	4.25	1142.0

4 Spark Transformations vs Actions

In Spark, operations can be divided into **transformations** and **actions**.

Transformations, such as `withColumn`, `filter`, `groupBy`, `agg`, `orderBy`, and `join`, do not process the data immediately. Rather, Spark tracks the steps that need to be performed. For example, `df.groupBy().agg()`, Spark creates a plan for the grouping and aggregation, but does not execute it yet. Actions tell Spark to actually execute this plan. These include `show()`, `count()`, and `toPandas()`. Therefore, when `demand_by_day_hour.show(20)` is called, Spark reads the parquet files and performs the required processing, including the Map Shuffle, and Aggregation (Reduce). The `.explain()` function, Section 5, shows an overview of the execution plan Spark has created.

5 Distributed Execution Evidence & Spark DAG

Using `demand_by_day_hour.explain("formatted")` shows the physical execution plan Spark generates for the pipeline in Section 3.

```
AdaptiveSparkPlan isFinalPlan=false
+- Sort [pickup_day_of_week ASC, pickup_hour ASC]
   +- Exchange range partitioning
      +- HashAggregate
         keys=[pickup_day_of_week, pickup_hour]
         functions=[count(1), avg(trip_miles), avg(trip_time)]
      +- Exchange hash partitioning
         keys=[pickup_day_of_week, pickup_hour]
         +- HashAggregate
            functions=[partial_count(1),
                       partial_avg(trip_miles),
                       partial_avg(trip_time)]
         +- Project [pickup_day_of_week, pickup_hour,
                    trip_miles, trip_time]
```

```
-- Scan parquet (pruned columns)
```

5.0.1 Reading the Physical Plan

Reading the plan from bottom to top shows the sequence of operations Spark intends to perform when executing the query. `Scan parquet` reads the required data from the parquet file, while `Project` represents the Map step from Section 3.1. The first `HashAggregate` performs a **partial aggregation**, in which Spark calculates counts and sums separately within each partition before transferring data between partitions. This is similar to a Hadoop combiner and reduces the amount of data that must be shuffled. `Exchange hash partitioning` performs the shuffle from Section 3.2, moving data so that rows with the same day/hour key are placed in the same partition. The second `HashAggregate` then performs the final Reduce step by combining the results from each partition to produce the final totals.

The `Exchange range partitioning` and `Sort` operations are a result of the `orderBy` operation. Although the aggregation results have already been calculated, they are still spread across different partitions and may not be in the correct order. Spark therefore redistributes the results between partitions and sorts them into the required overall order. This causes a second shuffle. Finally, `AdaptiveSparkPlan isFinalPlan=false` means that Spark may still adjust the execution plan while the query is running based on the actual data it processes. Each `Exchange` defines a new Spark job. Spark must complete the previous job before the next job can process its shuffle output.

5.0.2 Spark DAG Execution vs Hadoop MapReduce

Classic Hadoop MapReduce handles work through distinct Map and Reduce jobs. So, if the goal is to compute totals by day/hour and then order those totals, Hadoop typically needs two separate jobs. The first one aggregates the data and stores the result in HDFS. The second job then uses that output and carries out the sort. Spark, by contrast, can complete the same sequence within one query. It builds a Directed Acyclic Graph (DAG) that links the steps [5]. It first selects the data and does a partial aggregation, then performs the first shuffle, the final aggregation, and finishes with a second shuffle to order the results. Because of this, Spark can move intermediate outputs from one stage to the next without forcing each stage to be written to HDFS and reloaded as a fresh job.

As an example, imagine a single partition holding the values 10, 20 and 30 for the same day/hour. Spark's first `HashAggregate` can produce a partial sum of 60 before the shuffle occurs. Rather than transferring all three numbers, Spark can send just the partial value 60. Once the shuffle is complete, the second `HashAggregate` merges that value with partial results from other partitions to produce the final total. This kind of partial aggregation resembles a Hadoop combiner. Hadoop requires the programmer to define the combiner, and its execution is not guaranteed [6]. However, Spark can add partial aggregation automatically as part of query optimisation.

6 Results & Technical Interpretation

6.1 Demand by Day of Week and Hour

The results indicate a distinct daily demand cycle spanning all seven days. Demand is at its minimum from about 2am to 5am, then rises as the morning progresses and reaches stronger levels later in the day. Figure 1 in the Appendix shows a clear weekday morning surge from 7am to 9am and another evening surge from 5pm to 7pm, visible in the darker cells. The greatest weekday demand is recorded on Friday at 7pm, with 2,484,465 trips in the analysed processing set. The steady increase in weekday evening demand may be due to people commuting home or travelling after work.

A second peak appears around midnight on Friday, Saturday and Sunday, with Saturday midnight showing particularly high demand. Unlike the weekday evening peak, this increase is concentrated around the weekend and is most likely related late night social and entertainment activity than regular commuting. The difference between these patterns is important because it shows that high HVFHV demand is not associated with a single time period or type of travel. Instead, the results identify two main high demand periods.

6.2 Peak Hour Pickup Zones

To examine where demand is concentrated during the 5pm to 7pm peak, trips were grouped by pickup zone and joined with the NYC TLC zone lookup table. The results in Table 3 and Figure 2 in the Appendix show that demand is concentrated in a relatively small number of areas.

Table 3: Top five pickup zones by trip count during the 5pm–7pm peak.

Rank	Pickup zone	Borough	Trips
1	LaGuardia Airport	Queens	742,305
2	Midtown Center	Manhattan	676,856
3	Crown Heights North	Brooklyn	621,349
4	JFK Airport	Queens	609,358
5	West Chelsea/Hudson Yards	Manhattan	589,800

LaGuardia Airport records the highest demand with 742,305 trips, while JFK Airport also appears in the top five with 609,358 trips. This may indicate that airport related travel represents an important source of demand during the evening peak. Midtown Center and West Chelsea/Hudson Yards also rank among the busiest pickup zones, reflecting the high level of travel activity in these parts of Manhattan. Crown Heights North, however, ranks third and shows that high demand is not limited to Manhattan and the airports.

Overall, the geographic results show that evening demand is not evenly distributed across New York City. From an operational perspective, identifying these locations together with the peak time period could help inform where drivers should be positioned when demand is highest.

6.3 Monthly Trip Volume

The monthly analysis provides a long term view of how HVFHV demand changed over the chosen period. Selected values are shown in Table 4, while the complete trend is presented

in Figure 3 in the Appendix.

Table 4: Selected monthly trip volumes showing the COVID-19 decline and subsequent recovery.

Month	Trip count
2019-12	21,805,569
2020-05	6,079,705
2020-10	13,256,489
2022-10	19,278,299
2024-12	21,063,624

Trip counts drop significantly from 21.81 million in December 2019 to just 6.08 million in May 2020, which is approximately a 72% decrease. Demand then rebounded to 13.26 million trips by October 2020, but still stayed below the pre-pandemic levels. Subsequent data indicates a slow recovery, with trips rising to 19.28 million in October 2022 and then to 21.06 million by December 2024.

The magnitude of the May 2020 drop shows that the pandemic resulted in a substantial disruption in the HVFHV industry. More importantly, this result supports the finding from Part 1, even though a different data subset and Spark was used. The correspondence between the two analyses offers further support that the decline and the later rebound represent a real shift in HVFHV trip activity, rather than being caused by the sampling method used.

7 Business & Societal Interpretation

The primary stakeholders are HVFHV platforms and their drivers. They can make use of the day/hour demand patterns as well as the peak zone rankings to get a better idea of when demand is high and where it is concentrated. These patterns can help drivers position themselves more strategically instead of reacting after the demand surge has passed. NYC transport planners and regulators are another key stakeholder group. The two demand patterns found in this analysis (weekday evening demand and weekend late night demand) shows how transport demand changes with both the hour of day and the day of week.

The peak zone findings (Figure 2) can help platforms and drivers position vehicles ahead of the 5pm to 7pm surge, particularly near LaGuardia, JFK and other high demand areas. This could reduce unnecessary driving in search for passengers and improve vehicle availability when and where demand is high.

For city planners, the weekend late night peak clearly indicates that transport demand extends beyond conventional weekday hours, which implies that planning efforts that focus mostly on weekday rush hours may miss significant late night activity. This analysis could inform decisions around late night public transport, traffic management and service availability in areas with strong evening activity during the weekend.

However, this study only considers HVFHV trips and therefore does not represent overall transport demand in New York City, as taxis, public transport and private vehicles are excluded. The results also show patterns and associations rather than causal relationships. For example, although the weekend midnight peak may be associated with nightlife and social activity, the dataset does not state the purpose of the trip.

The monthly analysis uses 14 selected months rather than all 73 months in the processing

dataset. While this shows the broad COVID-19 decline and subsequent recovery, finer month to month changes may be missed. Finally, demand is summarised by hour and pickup zone. This offers a helpful high level picture, but it can also obscure fine details. For example, a brief spike inside a single hour, or demand focused in one specific area of a pickup zone, gets merged with nearby observations and may not appear in the aggregated output.

8 Conclusion

By processing roughly 245 million trip records with PySpark, this report examined how NYC HVFHV demand varies across hours of the day and days of the week. The data shows that weekday evenings, roughly from 5pm to 7pm have a high demand, and weekend late night demand is high near midnight. The location breakdown also revealed strong evening demand near major airports and in certain urban areas. On a monthly basis, the figures indicated a significant drop in trips during the COVID-19 period, after which activity slowly rebounded.

The Spark physical plan illustrates how the work was executed. Initially, Spark carries out part of the aggregation inside each partition. After that, it shuffles records sharing the same keys so the final values can be calculated. A further shuffle is needed before sorting the results. Rather than treating these steps as separate MapReduce jobs, Spark arranges them as stages in one Directed Acyclic Graph (DAG).

In summary, the findings suggest that HVFHV demand in New York City extends beyond the usual weekday rush hour. Both time and place influence when demand appears, which may help with driver placement and transport planning. However, this study only identifies when and where trips happen. It does not explain why certain patterns exist. In addition, this dataset only focuses on HVFHV trips and therefore, it does not capture all transportation activity throughout New York City.

9 References

- [1] New York City Taxi and Limousine Commission *TLC trip record data — high volume for-hire vehicle (HVFHV) trip records* <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page> Accessed: 27 August 2026 n.d.
- [2] NYC Open Data *Frequently asked questions* <https://opendata.cityofnewyork.us/faq/> Accessed: 27 August 2026 n.d.
- [3] Pillay, T. and Smit, R. *MIT 805 group project* https://github.com/Thigan/MIT_805_Group_Project GitHub repository, accessed 3 September 2026 2026.
- [4] Armbrust, M. *et al* “Spark SQL: Relational data processing in spark ”in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD ’15)* 2015: pp. 1383–1394. DOI: [10.1145/2723372.2742797](https://doi.org/10.1145/2723372.2742797)
- [5] Zaharia, M. *et al* “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing ”in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’12)* 2012.
- [6] White, T. *Hadoop: The Definitive Guide*, 4th. O’Reilly Media, Sebastopol, CA, 2015.

Appendix

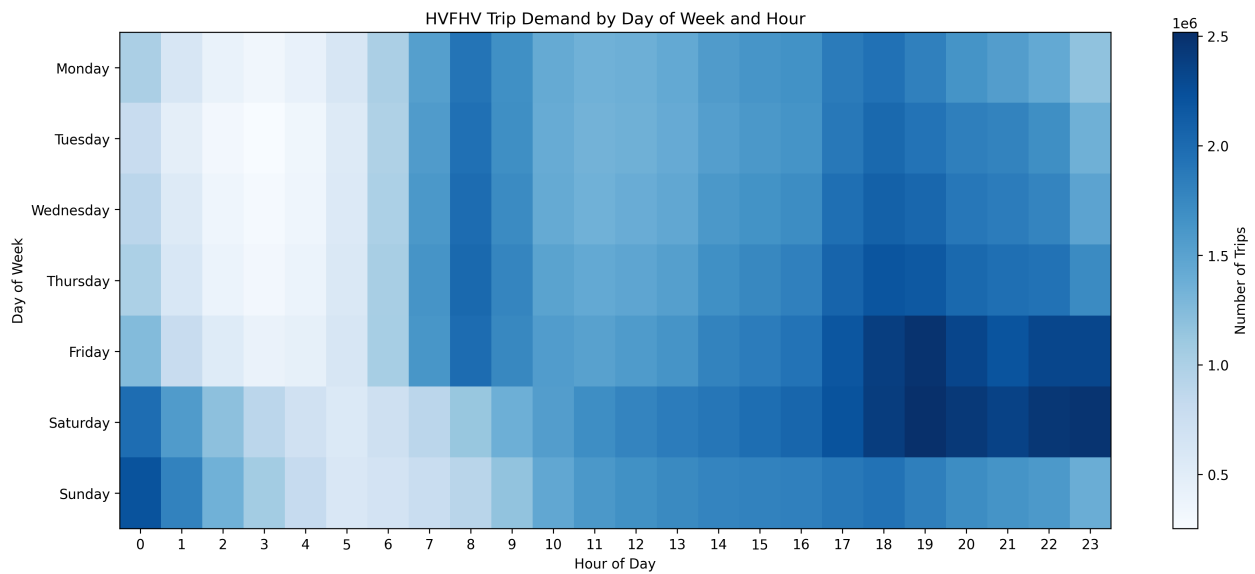


Figure 1: Trip counts by day of week and hour, computed via `demand_by_day_hour` (Section 4c). The darkest cells show that demand is highest during weekday evenings and around midnight on weekends.

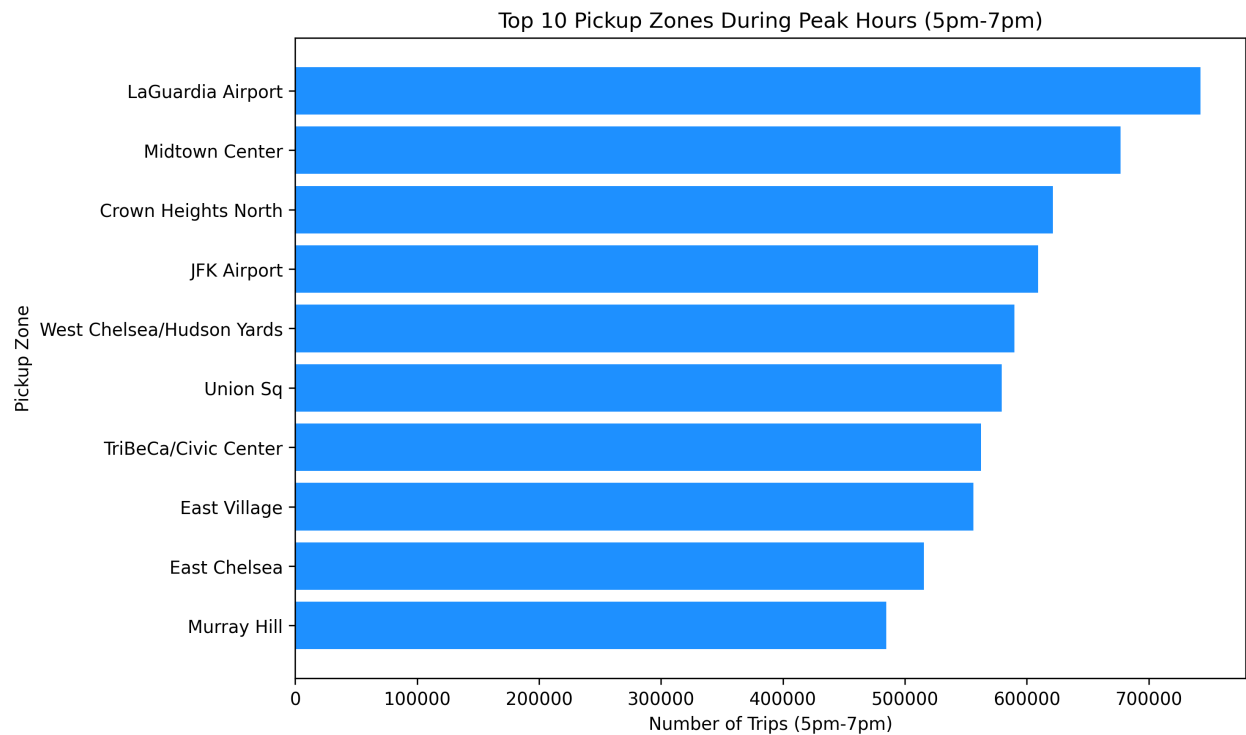


Figure 2: The ten pickup zones with the most trips during the 5pm–7pm peak (Section 4b). LaGuardia Airport leads.

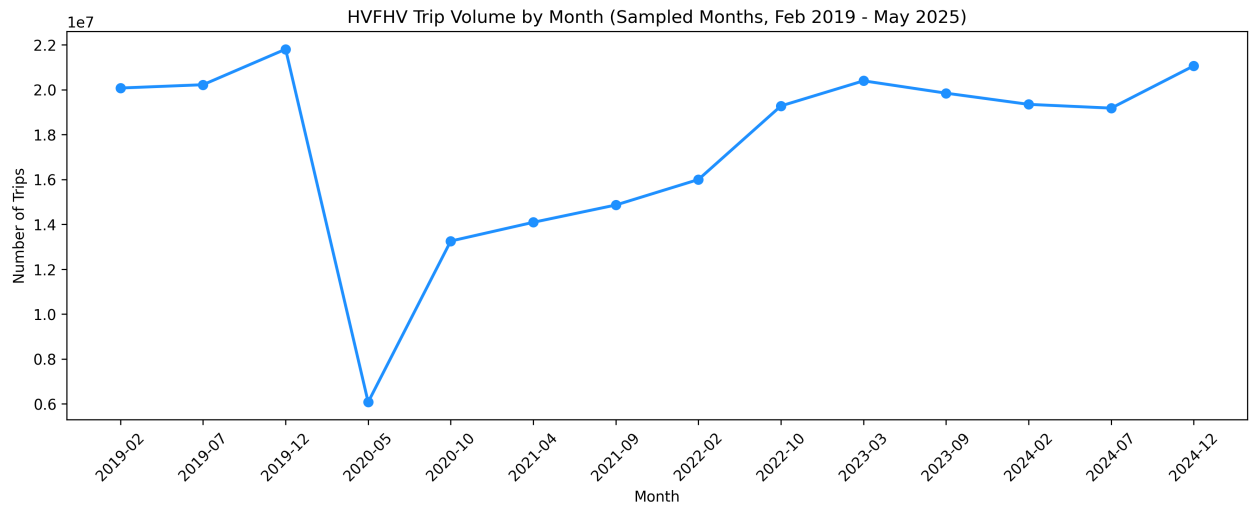


Figure 3: Total trip volume across the 14 evenly sampled months. The sharp drop around May 2020 corresponds to the COVID-19 pandemic’s effect on NYC transportation demand, supporting Part 1’s finding at full distributed scale.